

ชื่อ-สกุล: ธนพร นิมิตรหมื่นไวย B6612610 (โดนต์ B6610364)

บันทึกไฟล์เป็น Name LastName-Day01.pdf เช่น **Wichai Srisuruk-Day01.pdf**

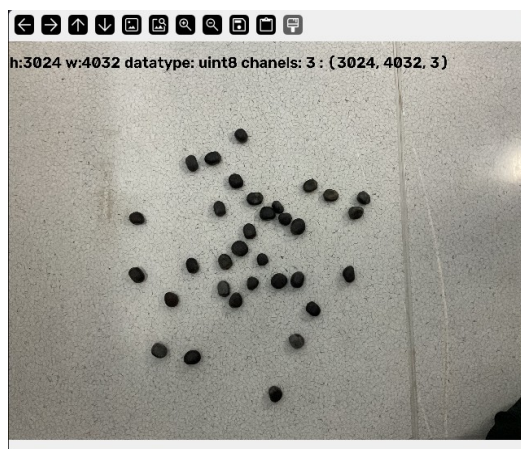
ส่งงาน ก่อนเลิกเรียน ก่อน 20260822-1700 ที่
<https://forms.gle/qkEVhifgsX682nzc8>

ส่งงาน รอบสมบูรณ์ ก่อน 20260828-0600 ที่
<https://forms.gle/qkEVhifgsX682nzc8>

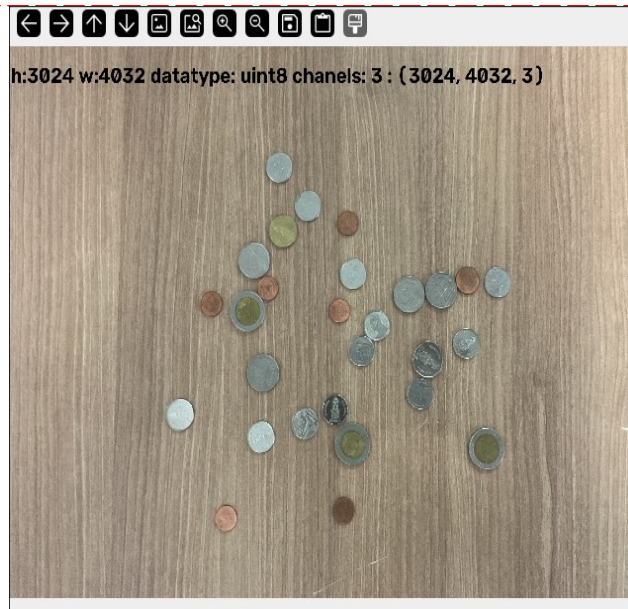
หัวข้อที่ 1: พื้นฐาน Machine Vision และการเตรียมภาพ

Q11: เขียนโปรแกรมอ่านภาพชิ้นงานจากกล้อง แสดงค่าความละเอียด (Resolution) และจำนวนช่องสัญญาณสี

Capture ผลการทำงาน



Capture ผลการทำงาน

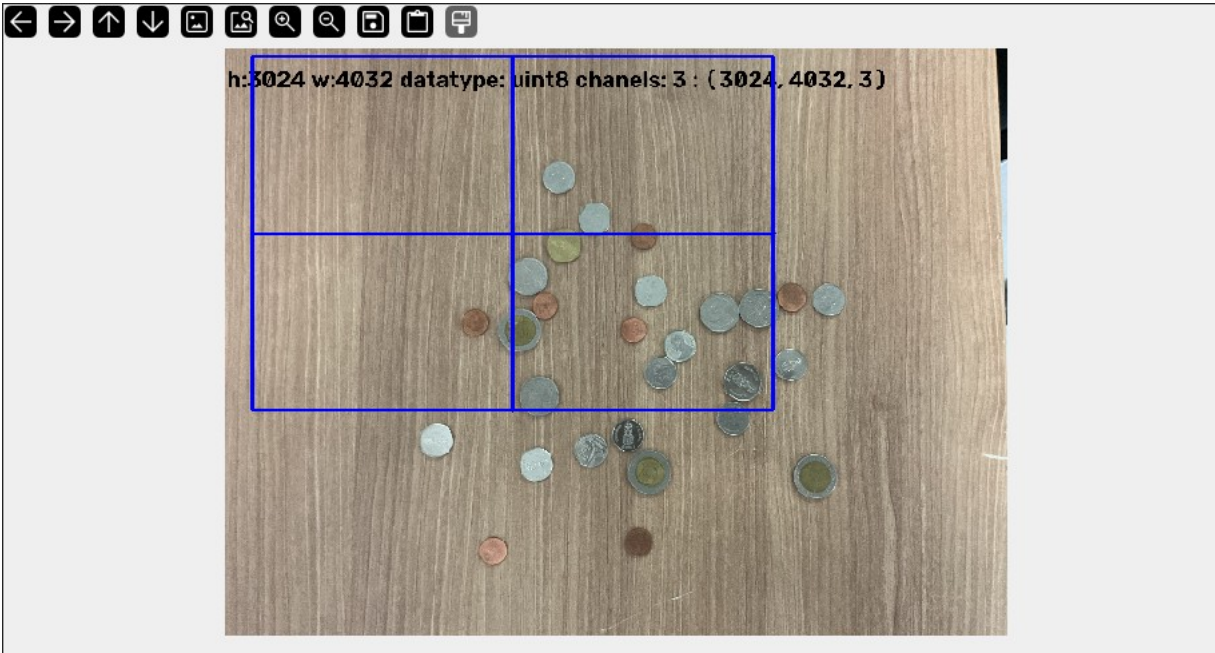


คำอธิบายผลการทำงาน

ดึงภาพ แล้วอ่าน shape attribute ของภาพ แล้วนำมาแสดงผลบนภาพ chanel datatype ความสูง และ ความกว้าง แล้วแสดงภาพออกมา 3 chanel หมายถึง เป็นแบบ RGB

Q12: ทดลองตัด ROI เฉพาะบริเวณที่ต้องการตรวจสอบ แล้วบันทึกเป็นไฟล์ภาพ
แยกต่างหาก

Capture ผลการทำงาน



Capture ผลการทำงาน



คำอธิบายผลการทำงาน

อ่านภาพ แล้ว ใช้ selectROI เพื่อเลือกขนาดที่จะตัดภาพ แล้วแสดงผลภาพ ใน window ใหม่

Q13: เปรียบเทียบภาพเดียวกันที่ถ่ายภายใต้ไฟส่องสว่างต่างชนิดกัน (Direct vs Diffuse) แล้วอภิปรายผลต่างของคุณภาพภาพ

คำอธิบาย

เวลาเราถ่ายรูปหรือมองสิ่งของรอบตัว สิ่งที่ทำให้เรามองเห็นวัตถุได้ก็คือ "แสง" ที่ตกกระทบวัตถุแล้วสะท้อนเข้าตาเรา แต่แสงที่สะท้อนออกมานั้นไม่ได้มีแบบเดียวนั้นแบ่งออกได้เป็นสองลักษณะใหญ่ ๆ คือ แสงตรง (Direct) กับ แสงกระจาย (Diffuse) สองอย่างนี้ทำให้ภาพที่เราได้มีหน้าตาต่างกัน และมีผลอย่างมากต่อวิธีที่คอมพิวเตอร์นำภาพไปประมวลผลต่อ ลองนึกภาพง่าย ๆ ว่าเรากำลังส่องไฟฉายไปที่ของสองชิ้น ชิ้นแรกเป็นกระจกเงา อีกชิ้นเป็นกระดาษด้าน ๆ เมื่อไฟตกกระทบกระจก เราจะเห็นจุดสว่างจ้าเป็นจุดเล็ก ๆ ตรงตำแหน่งที่แสงสะท้อนพอดี พอเราขยับหัวไปมา จุดสว่างนั้นก็เลื่อนตามไปด้วย นี่คือลักษณะของ แสงตรง คือแสงเดินทางมาชนแล้วแดงออกไปในทิศทางเดียวชัดเจน เหมือนลูกบอลกระทบพื้นแล้วแดงกลับตามมุมที่แน่นอน ทำให้เกิดจุดไฮไลต์วาว ๆ และเงาที่มีขอบคมชัด ในทางกลับกัน เมื่อไฟตกกระทบกระดาษด้าน แสงจะกระจายออกไปทุกทิศทางเท่า ๆ กัน ทำให้ผิวกระดาษดูสว่างสม่ำเสมอทั้งแผ่น ไม่ว่าเราจะขยับหัวไปมองมุมไหนก็เห็นความสว่างเท่ากัน นี่คือลักษณะของ แสงกระจาย ผิวแบบนี้ให้เงาที่นุ่มนวล ขอบเบลอ ไม่มีจุดวาวจ้าให้แสบตา วัตถุรอบตัวเราส่วนใหญ่ เช่น ผ้า ผืนหนัง หรือกำแพงปูน ล้วนสะท้อนแสงแบบกระจายทั้งนั้น ทีนี้พอมาถึงเรื่องการประมวลผลภาพด้วยคอมพิวเตอร์ ความต่างของสองอย่างนี้กลายเป็นเรื่องใหญ่ เพราะแสงตรงมักเป็น "ตัวสร้างปัญหา" จุดไฮไลต์ที่สว่างจ้าเกินไปมักทำให้ข้อมูลสีตรงนั้นหายไปหมด กลายเป็นสีขาวโพลนที่กู้คืนสีจริงไม่ได้ เหมือนถ่ายรูปย้อนแสงแล้วหน้าคนกลายเป็นดวงไฟ นอกจากนี้โปรแกรมที่พยายามเดารูปร่างหรือความลึกของวัตถุก็มักงงกับจุดวาวเหล่านี้ เพราะมันไม่ได้บอกอะไรเกี่ยวกับรูปร่างที่แท้จริงของผิวเลย นักพัฒนาจึงต้องมีเทคนิคพิเศษไว้ลบไฮไลต์หรือลดแสงจ้าออกก่อน ถึงจะทำงานต่อได้สะดวก ส่วนแสงกระจายนั้นตรงกันข้าม มันเป็น "เพื่อนที่ดี" ของการวิเคราะห์ภาพ เพราะความสว่างของผิวสัมพันธ์กับทิศทางของผิวและทิศทางของแสงอย่างตรงไปตรงมา ตรงไหนหันเข้าหาแสงก็สว่าง ตรงไหนหันหนีก็มีด งานหลายอย่าง เช่น การเดารูปร่างสามมิติจากความมืดสว่าง หรือการแยกว่าอะไรคือสีจริงของวัตถุกับอะไรคือเงา ล้วนตั้งอยู่บนสมมติฐานว่าผิวสะท้อนแสงแบบกระจาย เพราะมันคำนวณง่ายและคาดเดาได้ ด้วยเหตุนี้ นักวิจัยบางกลุ่มจึงคิดวิธี "แยก" แสงสองแบบนี้ออกจากกันในภาพเดียว โดยการฉายแสงที่มีลวดลายละเอียดลงบนฉาก แล้วสังเกตว่าส่วนไหนของภาพตอบสนองแบบแสงตรงที่สะท้อนครั้งเดียว กับส่วนไหนที่เกิดจากแสงกระจายและสะท้อนไปมาหลายรอบ เมื่อแยกได้แล้ว เราก็นำแต่ละส่วนไปใช้ประโยชน์ต่อได้ เช่น การจัดแสงใหม่ให้ภาพ หรือการวิเคราะห์

ว่าวัตถุทำจากวัสดุอะไร สรूपง่าย ๆ ก็คือ แสงตรงทำให้ภาพดูมีมิติ วาว และสมจริง แต่ยากต่อการประมวลผล ส่วนแสงกระจายทำให้ภาพดูเรียบนุ่มและวิเคราะห์ได้ง่ายกว่า การเข้าใจความต่างของทั้งสองอย่างจึงเป็นพื้นฐานสำคัญ ไม่ว่าเราจะทำงานด้านสร้างภาพสามมิติ ตกแต่งภาพถ่าย หรือสอนคอมพิวเตอร์ให้ "มองเห็น" โลกได้เหมือนที่ตาเรามองเห็น



Q14: ให้ออกแนวทางการใช้งาน Machine Vision กับงานที่ผู้เรียนเกี่ยวข้องหรือสนใจ

คำตอบ

ตรวจสอบคุณภาพ (Inspection) ตรวจสอบตำหนิบนชิ้นงาน เช่น รอยขีดข่วน รอยร้าว จุดสี หรือชิ้นส่วนที่ประกอบไม่ครบ ทำได้เร็วและแม่นยำมากกว่าคนมาก

วัดขนาด (Metrology) วัดความกว้าง ยาว เส้นผ่านศูนย์กลาง หรือระยะห่างของชิ้นงานแบบไม่ต้องสัมผัส เพื่อเช็คว่าอยู่ในค่าที่ยอมรับได้ไหม

อ่านรหัส (Identification) อ่านบาร์โค้ด QR code หรือตัวอักษรบนฉลาก (OCR) เพื่อติดตามชิ้นงานในสายการผลิตและการจัดการสต็อก

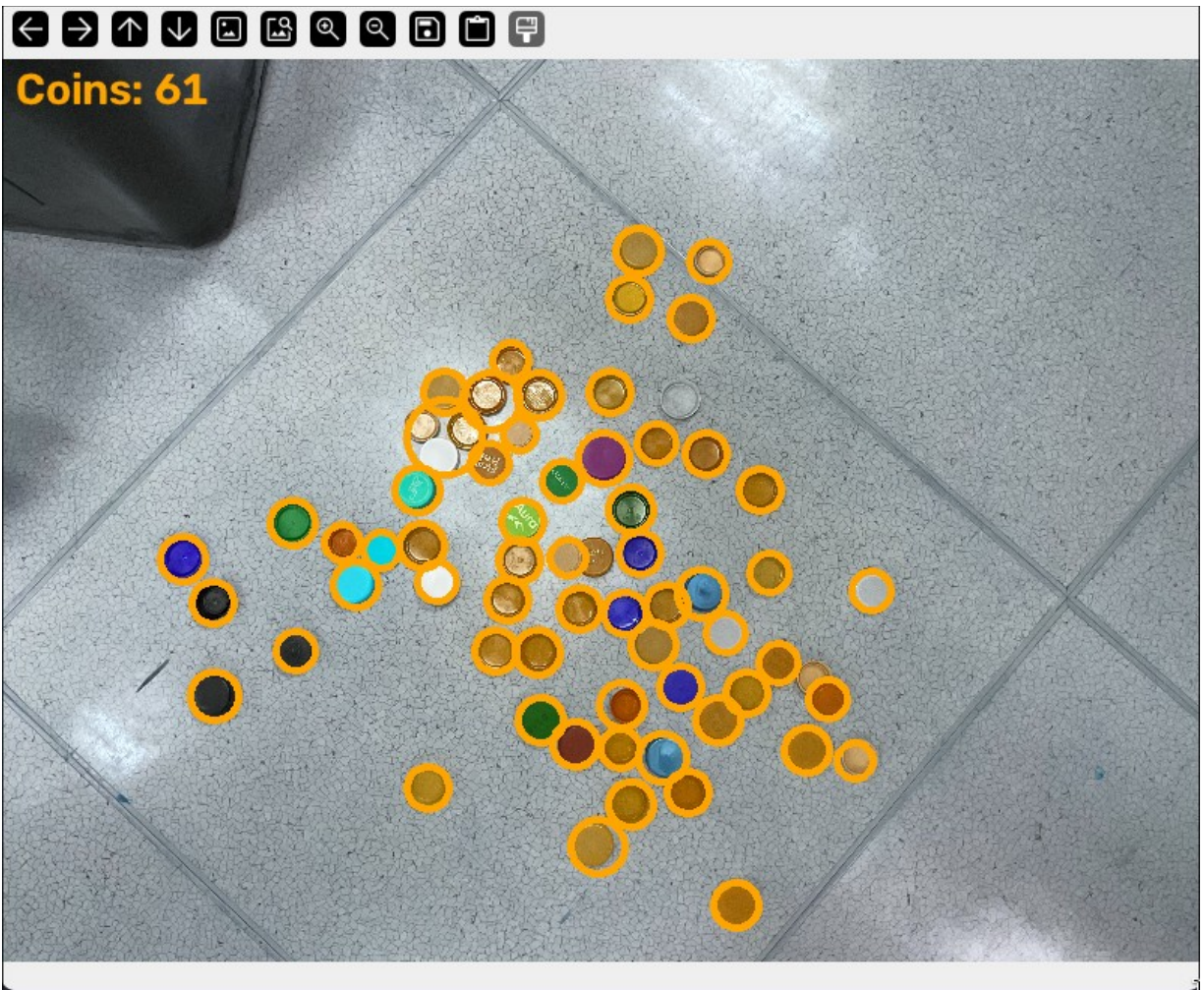
นำทางหุ่นยนต์ (Robot Guidance) บอกตำแหน่งและมุมของชิ้นงานให้แขนกลหยิบจับหรือประกอบได้ถูกต้อง แม้ชิ้นงานจะวางไม่เป็นระเบียบ

คัดแยกและนับ (Sorting/Counting) แยกชิ้นงานตามสี ขนาด หรือรูปทรง และนับจำนวนอัตโนมัติบนสายพาน

หัวข้อที่ 2: การประมวลผลภาพขั้นพื้นฐาน (OpenCV)

Q21: ถ่ายภาพเหรียญจำนวนหนึ่ง แล้วใช้โปรแกรมนับจำนวนวัตถุ โดยปรับค่า param1, param2, minRadius และ maxRadius ของ HoughCircles แล้ว สังเกตผลกระทบต่อจำนวนวัตถุที่ตรวจพบ

Capture ผลการทำงาน

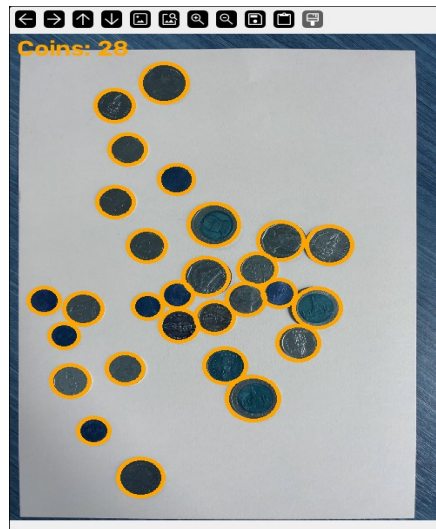


Capture ผลการทำงาน

Coins: 30



Capture ผลการทำงาน



คำอธิบายผลการทำงาน

โปรแกรมนี้เขียนขึ้นเพื่อให้คอมพิวเตอร์มองภาพถ่ายเหรียญแล้วนับว่ามีเหรียญอยู่ที่เหรียญ โดยใช้หลักการง่ายๆ คือ สอนให้คอมพิวเตอร์รู้จัก "วงกลม" ในภาพ เพราะเหรียญทุกเหรียญมีรูปร่างเป็นวงกลมเหมือนกัน

ขั้นแรก โปรแกรมจะเปิดไฟล์ภาพเหรียญขึ้นมา แล้วลดขนาดภาพให้เล็กลงเหลือความกว้าง 500 พิกเซล เพราะภาพต้นฉบับมีขนาดใหญ่เกินไป การลดขนาดจะช่วยทำให้คอมพิวเตอร์ประมวลผลได้เร็วขึ้น และเวลาลดขนาด โปรแกรมจะคำนวณความสูงให้ได้สัดส่วนเดียวกับภาพเดิม ไม่ทำให้ภาพบิดเบี้ยว

ต่อมา โปรแกรมจะแปลงภาพสีให้กลายเป็นภาพขาวดำ (เรียกว่าภาพเกรย์สเกล) เหตุผลคือ วิธีการหาวงกลมของคอมพิวเตอร์นั้นอาศัยความสว่างของแต่ละจุดในภาพ ไม่ได้ใช้สีสันทัน จึงตัดสีทิ้งเพื่อให้ประมวลผลง่ายขึ้น หลังจากนั้นโปรแกรมจะทำให้ภาพเบลอเล็กน้อยด้วยเทคนิคที่เรียกว่า Gaussian Blur ซึ่งเปรียบเทียบ

การเอานิวเซ็คกระจกที่มีรอยขีดข่วนเล็กๆ ออกไป การเบลอนี้จะช่วยลบสัญญาณรบกวนเล็กๆ น้อยๆ เช่น ลวดลายหรือรอยขีดข่วนเหรียญ ทำให้คอมพิวเตอร์ไม่สับสนและหาขอบของเหรียญได้แม่นยำขึ้น

จากนั้นเป็นขั้นตอนสำคัญที่สุด คือการใช้ฟังก์ชันที่ชื่อว่า HoughCircles ซึ่งเป็นเครื่องมือที่ถูกออกแบบมาโดยเฉพาะสำหรับค้นหาวงกลมในภาพ หลักการทำงานของมันเป็นคือ มันจะสแกนหาเส้นขอบในภาพทั้งหมด แล้วลองตรวจสอบดูว่าเส้นขอบเหล่านั้นโค้งเป็นวงกลมหรือไม่ ถ้าพบรูปร่างที่โค้งจนครบวงและมีขนาดรัศมีอยู่ในช่วงที่เรากำหนดไว้ มันจะบันทึกไว้ว่า "ตรงนี้มีวงกลมหนึ่งวง" พร้อมทั้งบอกตำแหน่งจุดศูนย์กลางและขนาดรัศมีของวงกลมนั้นด้วย

การจะให้ HoughCircles หาเหรียญได้ถูกต้อง เราต้องบอกมันหลายอย่าง เช่น รัศมีของวงกลมที่เล็กที่สุดและใหญ่ที่สุดที่ยอมรับได้ เพราะถ้าไม่กำหนดขอบเขตนี้ มันอาจไปหาวงกลมที่ไม่ใช่เหรียญ หรือมองเหรียญเป็นวงกลมผิดขนาดได้ นอกจากนี้ยังต้องกำหนดระยะห่างขั้นต่ำระหว่างจุดศูนย์กลางของวงกลมสองวง เพื่อป้องกันไม่ให้มันตรวจจับเหรียญเดียวกันซ้ำหลายครั้ง ค่าตัวเลขเหล่านี้ในโปรแกรมได้ถูกทดลองปรับจนกระทั่งได้ผลลัพธ์ที่ตรงกับจำนวนเหรียญจริงในภาพ ซึ่งก็คือ 28 เหรียญ

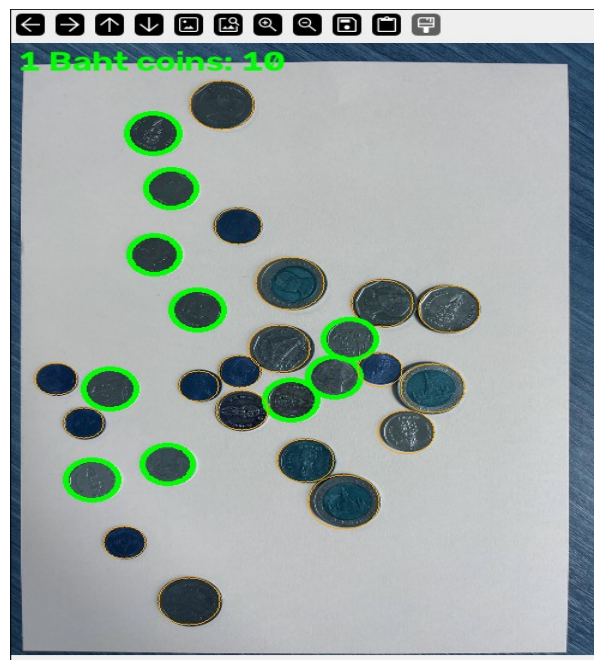
หลังจากที่โปรแกรมหาวงกลมเจอครบทุกวงแล้ว มันจะวนลูปเพื่อวาดเส้นวงกลมสีส้มทับลงไปบนตำแหน่งของเหรียญแต่ละเหรียญในภาพสี เพื่อให้เราเห็นด้วยตาว่าคอมพิวเตอร์เจอเหรียญตรงไหนบ้าง แล้วยังเขียนตัวหนังสือบอกจำนวนเหรียญทั้งหมดที่นับได้ไว้ที่มุมภาพด้วย

สุดท้ายโปรแกรมจะเปิดหน้าต่างแสดงผลขึ้นมาสามครั้งตามลำดับ เริ่มจากภาพขาวดำ ตามด้วยภาพที่เบลอแล้ว และปิดท้ายด้วยภาพสีที่มีวงกลมล้อมรอบเหรียญพร้อมตัวเลขจำนวนเหรียญ เพื่อให้ผู้ใช้เห็นขั้นตอนการทำงานทั้งหมด ตั้งแต่ต้นจนจบว่าคอมพิวเตอร์แปลงภาพไปที่ละขั้นอย่างไรก่อนจะสรุปผลลัพธ์สุดท้ายออกมา

Advance21:

วิธีการทำงานและข้อจำกัด: OpenCV ไม่สามารถอ่านข้อความบนขนาดเล็กอย่าง “1 บาท” บนเหรียญได้ ดังนั้นวิธีนี้จึง ไม่ใช้การรู้จำชนิดของเหรียญ (denomination recognition) อย่างแท้จริง แต่เป็นการระบุเหรียญ 1 บาท จากการจัดกลุ่มตาม สีและขนาด แทน:

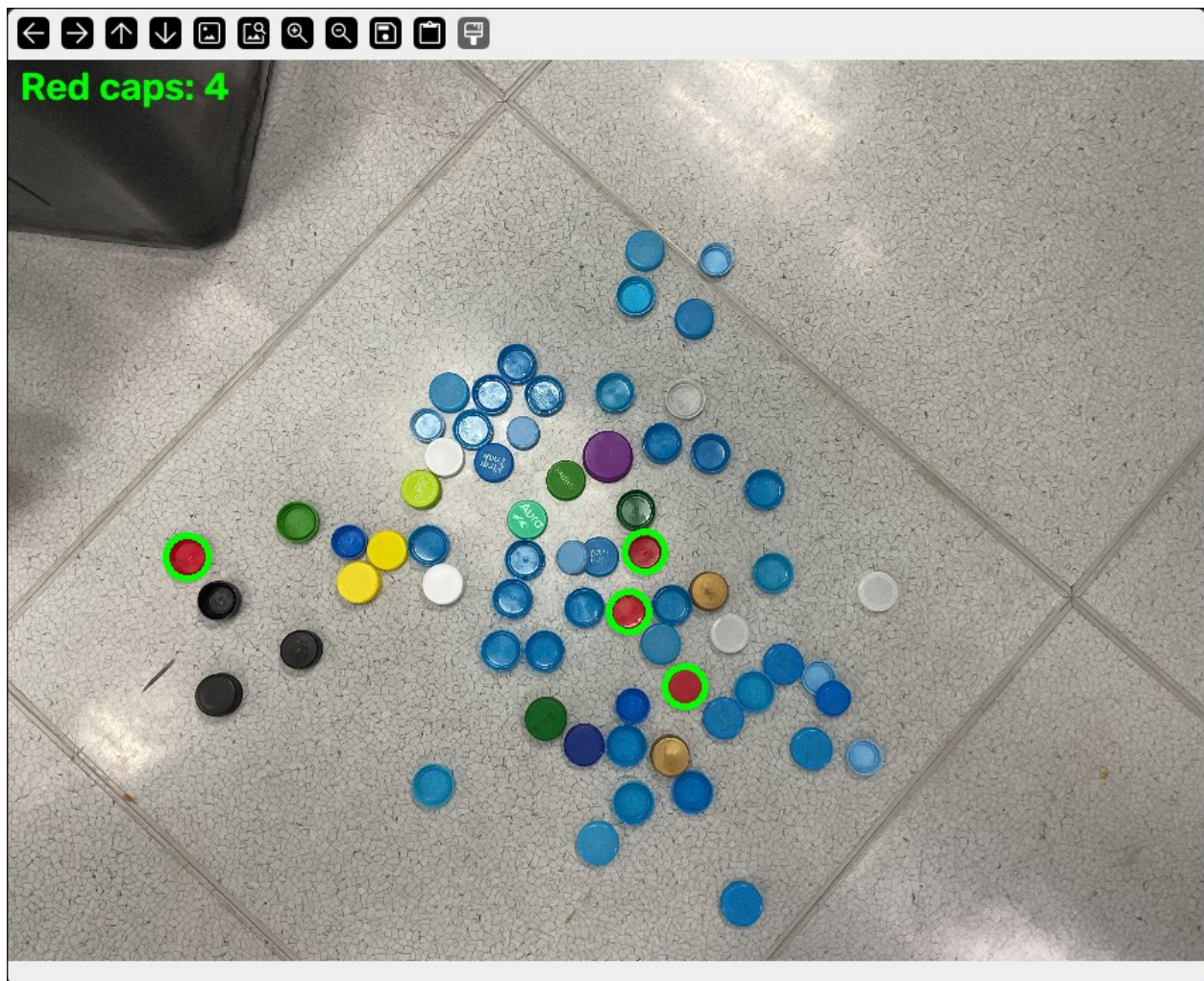
1. ตรวจจับเหรียญทั้งหมดด้วย HoughCircles (เช่นเดียวกับ q21.py)
2. สำหรับเหรียญแต่ละเหรียญ คำนวณค่าสี HSV เฉลี่ยจากบริเวณตรงกลางเหรียญ
3. จากการวัดรัศมีและค่าสี HSV ของเหรียญทุกเหรียญในภาพนี้ พบว่าเหรียญ 1 บาทจริง ๆ อยู่ในกลุ่มที่มีค่าค่อนข้างสม่ำเสมอและแยกออกจากกลุ่มอื่นได้ชัดเจน ได้แก่ รัศมี 21–23 px, saturation 50–95 และ hue 15–28 ซึ่งแตกต่างจากเหรียญทองแดงที่มี saturation สูงกว่า (160–230+) และเหรียญ 10 บาทแบบสองสีที่มี saturation สูงกว่า (150–190+) และมีรัศมีใหญ่กว่า
4. เหรียญที่อยู่ในช่วงค่าดังกล่าวจะถูกนับและตีกรอบด้วย สีเขียว ส่วนเหรียญอื่น ๆ จะยังคงตีกรอบด้วย สีส้ม



advance22:

แทนที่จะใช้ `HoughCircles` (ซึ่งเหมาะกับการตรวจจับ รูปร่าง มากกว่าการตรวจจับ สี) วิธีนี้ใช้การแบ่งส่วนภาพตามสีในระบบ HSV:

1. แปลงภาพเป็น HSV และกำหนดช่วงค่าสำหรับตรวจจับสีแดง (ต้องใช้ 2 ช่วง เนื่องจากสีแดงอยู่บริเวณรอยต่อของค่า hue ที่ $0^\circ/180^\circ$)
2. ทำความสะอาด mask ด้วย morphological opening/closing เพื่อลด noise
3. ค้นหา contours แล้วกรองด้วย พื้นที่ (area) เพื่อตัดจุดเล็ก ๆ ออก และกรองด้วย ความเป็นวงกลม (circularity) เพื่อตัดวัตถุที่ไม่กลม เช่น แสงสะท้อนสีแดง
4. วาดวงกลมสีเขียวพร้อมนับจำนวนบนฝาสีแดงที่ตรวจพบแต่ละอัน



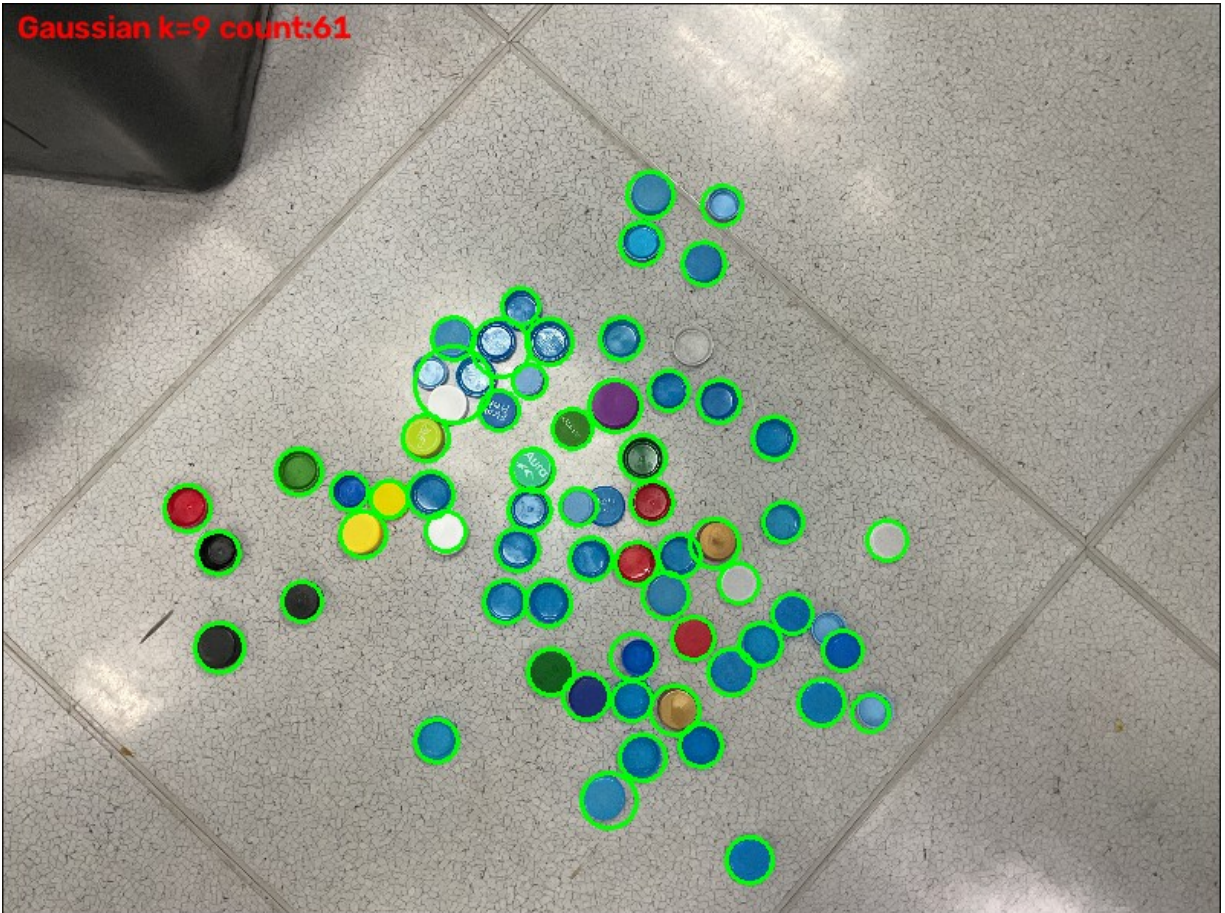
advance23:

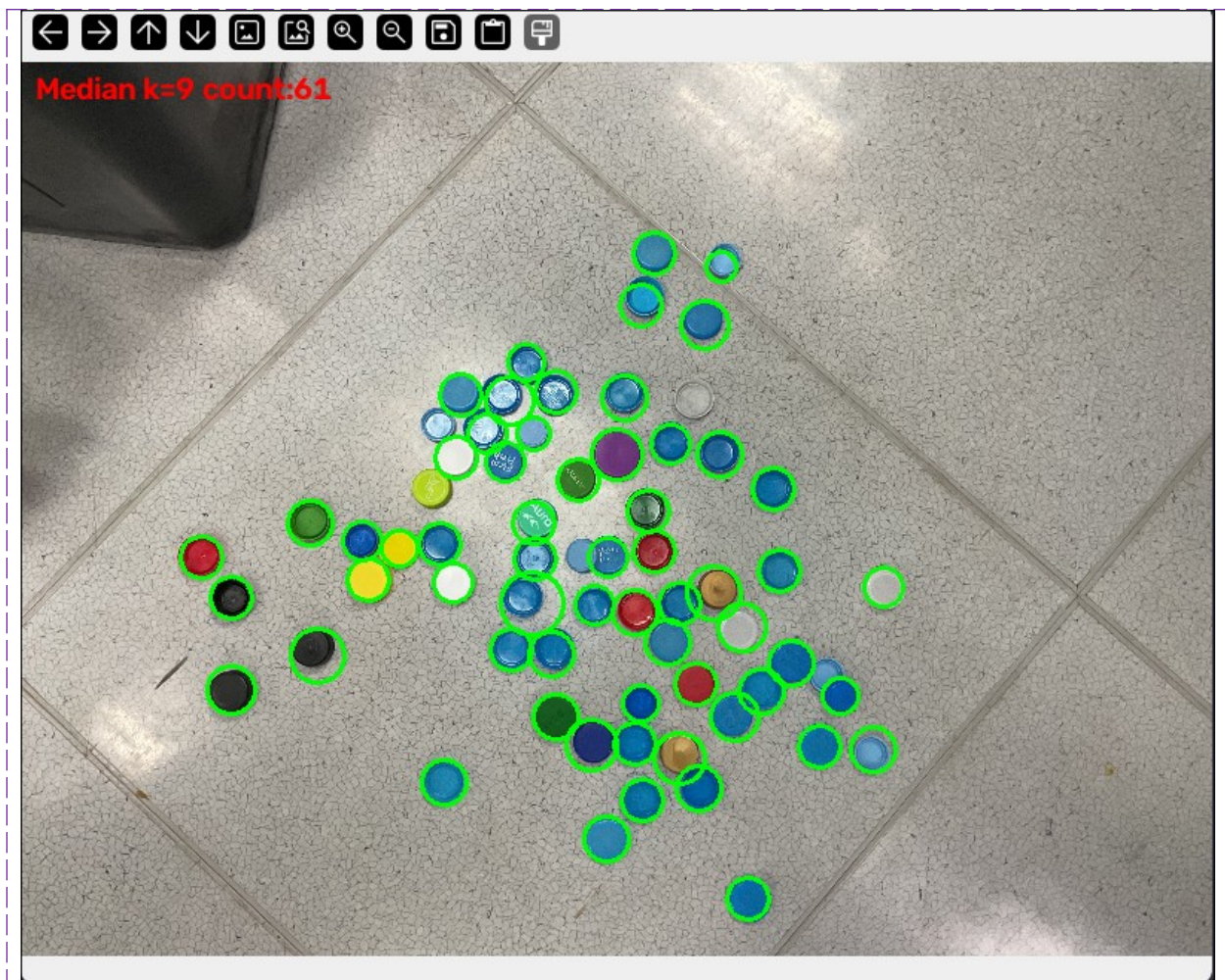
Yellow caps: 2



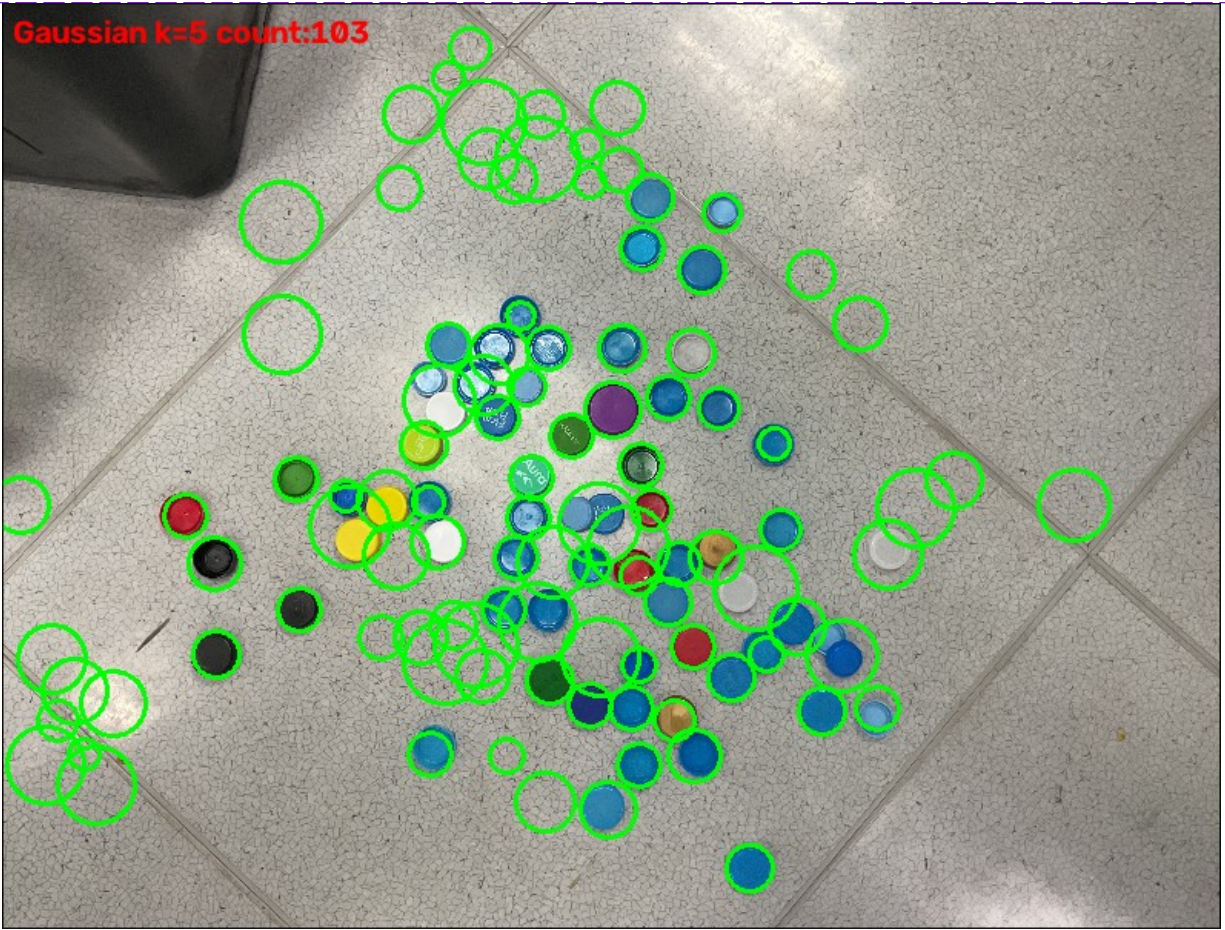
1

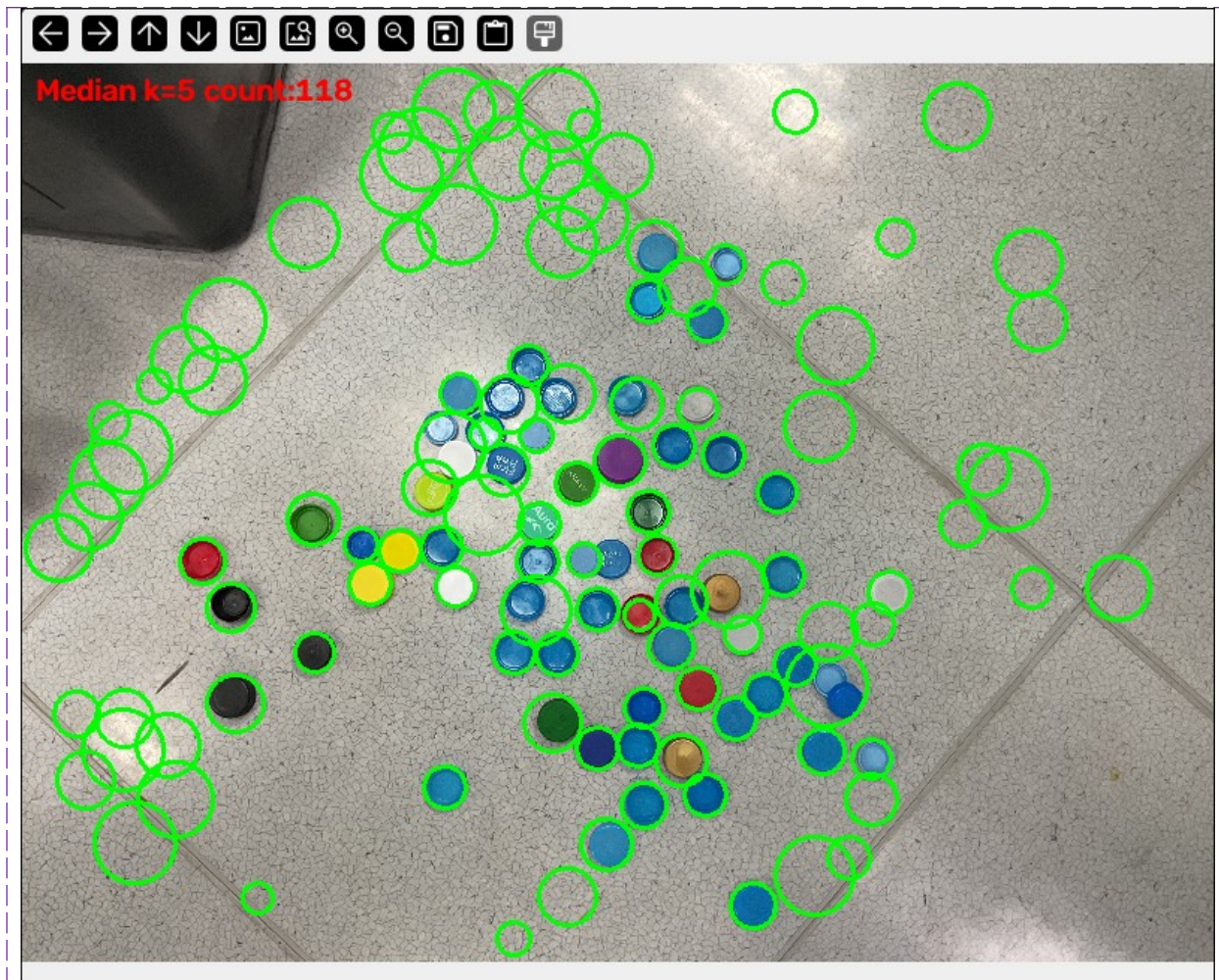
Q23: จากไปป์ไลน์ตัวอย่าง เปรียบเทียบผลของการทำ Median Blur กับ Gaussian Blur แล้ววิเคราะห์ความแตกต่างของผลลัพธ์การนับวัตถุ Capture ผลการทำงาน





Capture ผลการทำงาน



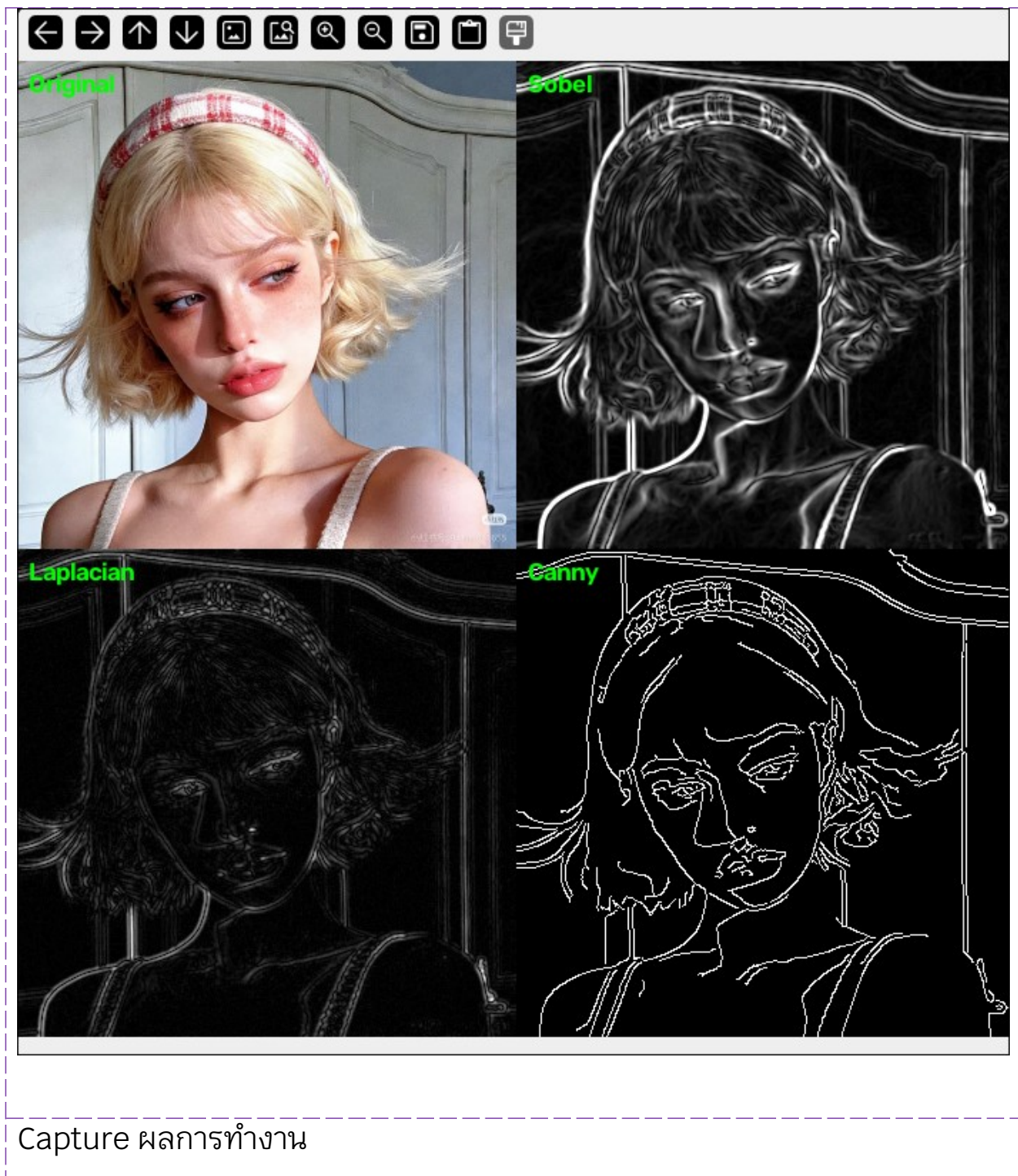


Median Blur รักษาขอบวัตถุจริงได้คมกว่า Gaussian Blur แต่แลกมาด้วยการกำจัด texture noise ที่เป็นลวดลายต่อเนื่องได้ไม่ดีเท่า ทำให้เมื่อใช้ kernel เล็กเกินไป Median Blur มีแนวโน้มสร้าง false positive มากกว่า Gaussian Blur ในภาพที่มีพื้นหลังเป็นลวดลาย (เช่นกระเบื้อง) แต่เมื่อปรับ kernel ให้ใหญ่พอเหมาะกับขนาดของ noise ทั้งสองวิธีจะให้ผลการนับที่แม่นยำใกล้เคียงกัน — สรุปคือ การเลือกชนิดของ blur สำคัญน้อยกว่าการเลือกขนาด kernel ให้เหมาะสมกับลักษณะพื้นหลังของภาพ

Q22: เลือกภาพจากอินเทอร์เน็ตจำนวน 3 ภาพ (X1, X2, X3) แต่ละภาพให้เปรียบเทียบผลลัพธ์การหาขอบระหว่าง Sobel, Laplacian และ Canny บนภาพชิ้นงานเดียวกัน

Capture ผลการทำงาน





Capture ผลการทำงาน



Original

Sobel

Laplacian

Canny

คำอธิบายผลการทำงาน

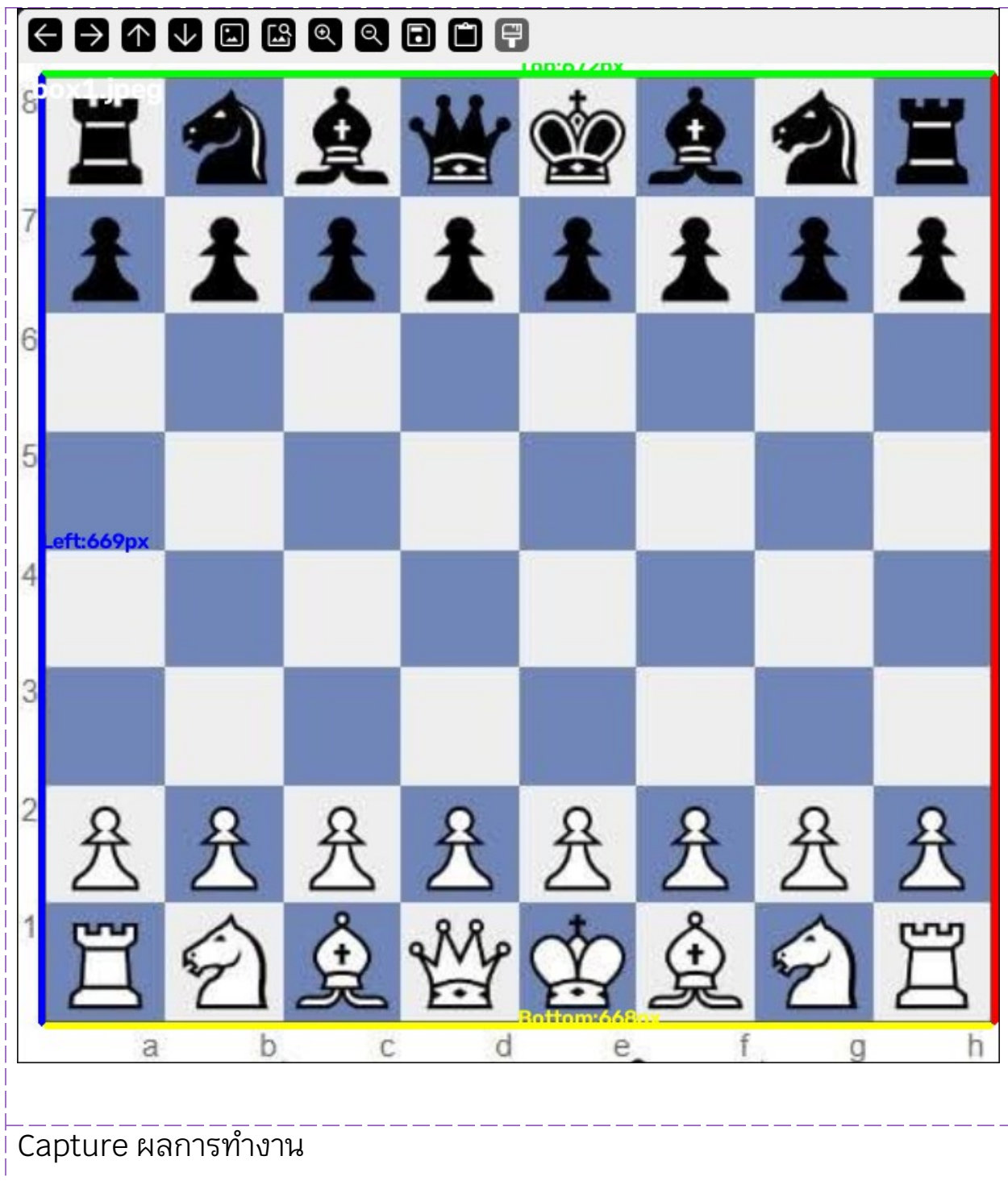
Sobel — ใช้ อนุพันธ์อันดับที่ 1 (1st-derivative) เพื่อคำนวณขนาดของ Gradient โดยรวมทิศทาง X และ Y ด้วย `cv.magnitude` ผลลัพธ์เป็นขอบที่หนาและสว่าง โดยแสดงระดับความแรงของ Gradient เป็นภาพโทนเทา แต่ค่อนข้างไวต่อ Noise และ Texture เช่น เส้นผมจะปรากฏขึ้นอย่างชัดเจน

Laplacian — ใช้ อนุพันธ์อันดับที่ 2 (2nd-derivative) และตรวจจับขอบในทุกทิศทางพร้อมกันด้วย Kernel เดียว ผลลัพธ์จะ จางและบางกว่า Sobel แต่มี Noise มากกว่า และไวต่อรายละเอียดพื้นผิวขนาดเล็ก จึงดูไม่สะอาดเมื่อใช้เพียงอย่างเดียว

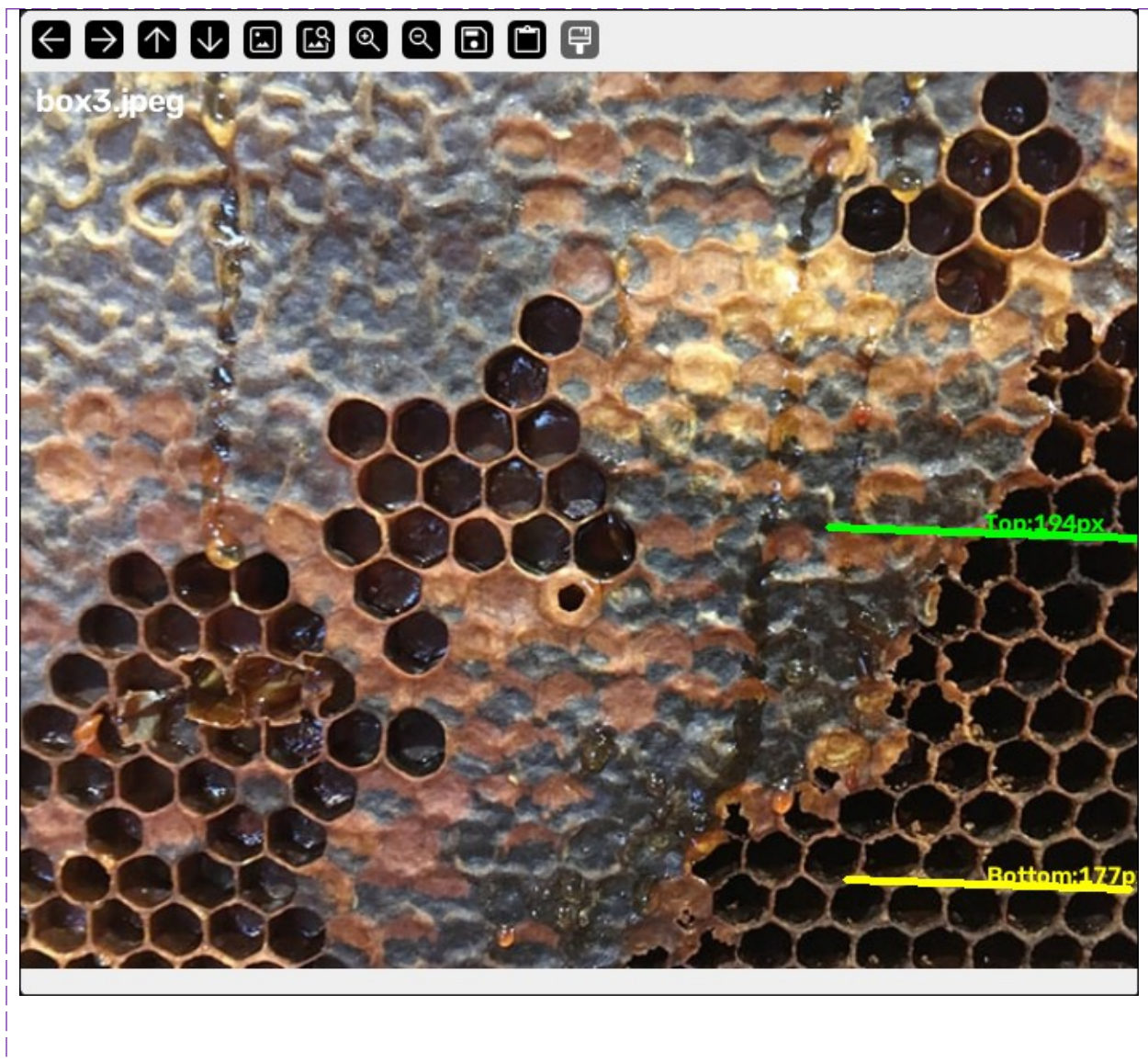
Canny — เป็นอัลกอริทึมแบบหลายขั้นตอน ประกอบด้วย Gradient + Non-Maximum Suppression + Hysteresis Thresholding โดยใช้ค่า Threshold 50, 150 ผลลัพธ์เป็นเส้นขอบที่ สะอาด บาง และต่อเนื่องที่สุด จึงเหมาะที่สุดสำหรับการสร้าง Silhouette หรือ Edge Map ที่ต้องการความชัดเจน เพราะสามารถตัดขอบที่อ่อนหรือเกิดจาก Noise ซึ่ง Sobel และ Laplacian ตรวจจับขึ้นมาได้ออกไปได้

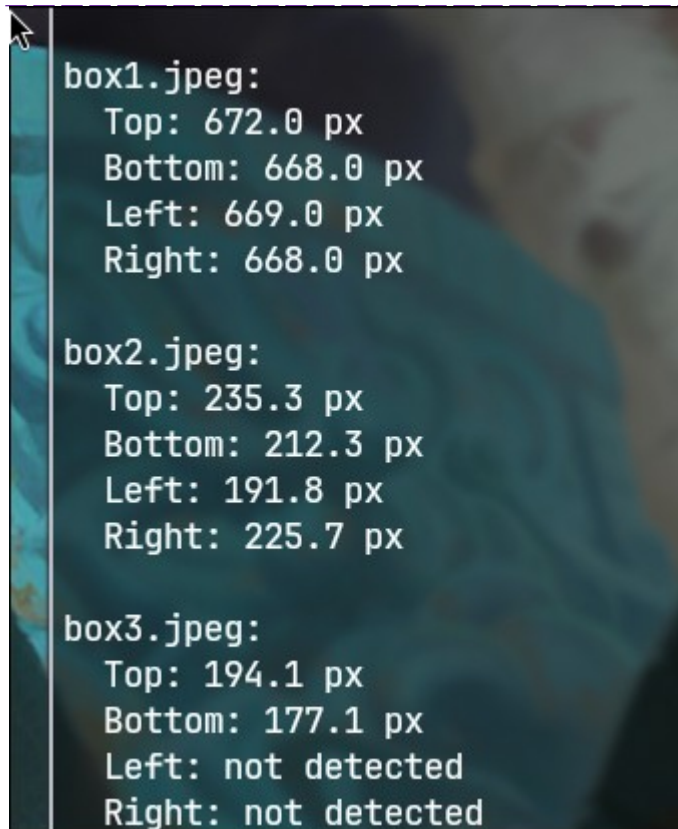
Q24: เลือกภาพจากอินเทอร์เน็ตจำนวน 3 ภาพ (Y1, Y2, Y3) แต่ละภาพให้ใช้ HoughLinesP ตรวจสอบหาขอบแนวตรงของชิ้นงานสีเหลี่ยม แล้วคำนวณความยาวของแต่ละด้านเป็นหน่วยพิกเซล

Capture ผลการทำงาน









คำอธิบายผลการทำงาน

box1.jpeg (กระดานหมากรุก) — ผลลัพธ์แม่นยำมาก ตรวจพบขอบทั้ง 4 ด้านของกระดานถูกต้องครบ:

- Top: 672px, Bottom: 668px, Left: 669px, Right: 668px

ความยาวทั้ง 4 ด้านใกล้เคียงกันมาก (สมเหตุสมผล เพราะกระดานหมากรุกเป็นสี่เหลี่ยมจัตุรัสจริง) เหตุผลที่ได้ผลดีคือใช้หลักการ "เลือกเส้นที่อยู่ นอกสุด ในแต่ละทิศทาง" แทนที่จะเลือก "เส้นที่ยาวที่สุด" — เพราะลายตารางหมากรุกทำให้เส้นแบ่งช่องภายใน (internal grid lines) ก็มีความยาวเกือบเท่าเส้นขอบนอกเช่นกัน (เนื่องจากสีสลับดำ-ขาวทำให้เกิดคอนทราสต์ตลอดทั้งแถว) ถ้าเลือกจาก "ยาวที่สุด" อย่างเดียวจะลุ่มได้เส้นแบ่งช่องภายในผิดๆ

box2.jpeg (ตึก) — ผลลัพธ์ไม่แม่นยำ: เส้น "Left" ไปตกที่ขอบอาคารข้างเคียง (ไม่ใช่ตึกเป้าหมาย), เส้น "Top" เป็นแค่ขอบหน้าต่างชั้นล่าง ไม่ใช่แนวหลังคา, เส้น

"Bottom" คือขอบถนน/ทางเท้า ไม่ใช่ฐานอาคาร และเส้น "Right" จับได้แค่บางส่วนของมุมตึกจริง ไม่ครอบคลุมสูง — สาเหตุคือภาพนี้ถ่ายตึกแบบ perspective (มุมเอียง) ทำให้ตึกไม่ใช่สี่เหลี่ยมที่ตรงแนวแกนภาพจริงๆ และมีเส้นคู่แข่งจำนวนมาก (ขอบหน้าต่าง, ขอบพื้นแต่ละชั้น, อาคารข้างเคียง) ที่ยาว/อยู่นอกสุดกว่าขอบอาคารจริงในบางทิศทาง วิธีนี้ใช้ไม่ได้ดีกับภาพวัตถุ 3 มิติที่ถ่ายมุมเอียงแบบนี้

box3.jpeg (รังผึ้ง) — ตรวจพบ ไม่มีสี่เหลี่ยมอยู่จริง ตามคาด: พบเฉพาะเส้นแนวนอนหลอกๆ 2 เส้น (จากรอยต่อของลายรังผึ้ง ไม่ใช่ขอบวัตถุจริง) และไม่พบเส้นแนวตั้งเลย ("Left"/"Right": not detected) — เป็นผลลัพธ์ที่ถูกต้องแล้ว เพราะภาพนี้ไม่มีชิ้นงานสี่เหลี่ยมให้ตรวจจับ